Università di Genova

UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

ARTIFICIAL INTELLIGENCE FOR ROBOTICS 2

Warehouse Robotics – Conveyor Belt Dynamics

Author:
Luca Allegri

Student ID:
s7905713

Professors:
Fulvio Mastrogiovanni

June, 2026

1 Introduction

This report focuses on the modelling of a warehouse logistics scenario using the Planning Domain Definition Language and PDDL+.

The core of this project is the orchestration of mobile manipulator robots operating in close coordination with a network of autonomous conveyor belts to transport packages from initial storage locations to designated delivery stations. In these environments, robots are responsible for manipulation tasks such as picking, loading, unloading, and delivering packages, while conveyor belts provide autonomous transportation between locations.

The work investigates how conveyor belts can be approximated through discrete transitions in classical PDDL and later represented more realistically through continuous processes and events in PDDL+.

2 Domain Characteristics and Modelling Guidelines

The warehouse is represented as a graph of storage areas, loading/unloading zones, conveyor segments, intermediate nodes and delivery stations. The main modelling requirement is to keep a clear separation between agent-controlled and environment-driven dynamics:

- **Robot-controlled actions**: navigation and picking, loading, retrieving, grasping and delivery packages in specific zones
- **Conveyor dynamics**: package transport performed by the conveyor infrastructure

3 Q1 – Basic PDDL Model

In the first part of the project, the warehouse scenario is modelled using classical PDDL. Since standard PDDL does not provide native support for continuous autonomous dynamics, the behavior of conveyor belts is approximated through a sequence of discrete transitions.

The domain has been designed to support both situations in which conveyor usage is simply advantageous and scenarios in which conveyors become strictly necessary to reach otherwise inaccessible areas of the environment.

3.1 Domain Modelling

The domain uses three object types: robot, package and location. The **predicate** set is organized to separately model navigation, manipulation, and conveyor interactions.

Table 1: Predicates in the basic PDDL domain.

Category	Predicates
Navigation	`connected(?from ?to)`, `robot-at(?r ?l)`
Manipulation	`package-at(?p ?l)`, `holding(?r ?p)`, `handempty(?r)`
Conveyor	`belt-connected(?from ?to)`, `on-belt(?p)`, `belt-free(?l)`
Zones	`load-zone(?l)`, `unload-zone(?l)`, `delivery-zone(?l)`

The **action** model includes move, pick, place, load-conveyor, retrieve and conveyor-step. In particular, the conveyor-step action is the core mechanism used to approximate conveyor dynamics in classical PDDL. Instead of modelling continuous movement, the package progresses discretely from one belt segment to the next through planner-generated actions.

3.2 Optional Conveyor Environment

In the first problem configuration, conveyor usage is optional. The warehouse topology still allows the robot to manually transport packages to their destinations using only navigation and manipulation actions. However, conveyor belts provide a potentially cheaper and more efficient transportation alternative reducing the total metric cost. As shown in Figure 1, the environment contains:

- one mobile robot (r1);

- two packages (pck1, pck2);
- two conveyor systems:
  1. a top conveyor connecting l2 → l3 → l4;
  2. a bottom conveyor connecting l9 → l10 → l11 → l12.

The robot starts in the central area of the warehouse, while packages are initially positioned in storage1 and storage3. The goal requires delivering: pck1 to delivery1 and pck2 to delivery4.

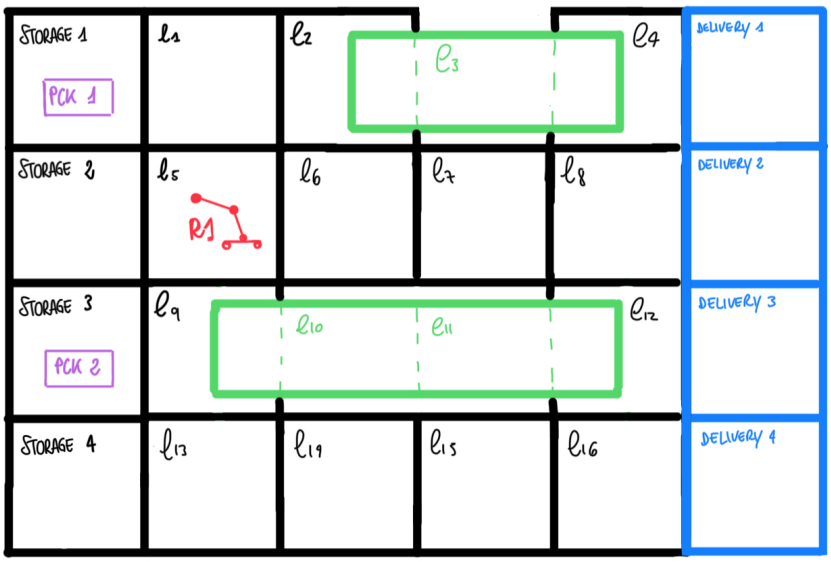

Figure 1: Environment with optional use of Conveyor Belts

Results Analysis – Optional Conveyor Environment

In this scenario, the warehouse topology still allows the robot to reach all relevant storage and delivery locations through standard navigation. Therefore, the conveyor belts are not strictly required to solve the task, but they may provide a cheaper alternative. Two different plans were obtained for this scenario.
The first solution has:

```
move r1 l5 l9
 move r1 l9 storage3
  pick pck2 r1 storage3
   move r1 storage3 l9
    move r1 l9 l5
     move r1 l5 l6
      move r1 l6 l7
       move r1 l7 l8
        move r1 l8 delivery2
         move r1 delivery2 delivery3
          move r1 delivery3 delivery4
           place pck2 r1 delivery4
            move r1 delivery4 delivery3
             move r1 delivery3 delivery2
              move r1 delivery2 l8
               move r1 l8 l7
                move r1 l7 l6
                 move r1 l6 l5
                  move r1 l5 l1
                   move r1 l1 storage1
                    pick pck1 r1 storage1
                     move r1 storage1 l1
                      move r1 l1 l5
                       move r1 l5 l6
                        move r1 l6 l7
                         move r1 l7 l8
                          move r1 l8 l4
                           move r1 l4 delivery1
                            place pck1 r1 delivery1
```

Figure 2: Optional non-optimal plan

```
Plan-Length:29
Metric (Search):54.0
Planning Time (msec): 40
Heuristic Time (msec): 18
Search Time (msec): 37
Expanded Nodes:41
States Evaluated:68
Number of Dead-Ends detected:3
Number of Duplicates detected:52
```

Figure 3: Optional non-optimal costs

This solution is very fast to compute because the planner follows a direct and simple strategy. The search space is small, as shown by the low number of expanded nodes and evaluated states. However, the resulting cost is not the best one, because the robot performs many navigation actions while physically carrying the packages.
The second solution for the optional environment has:

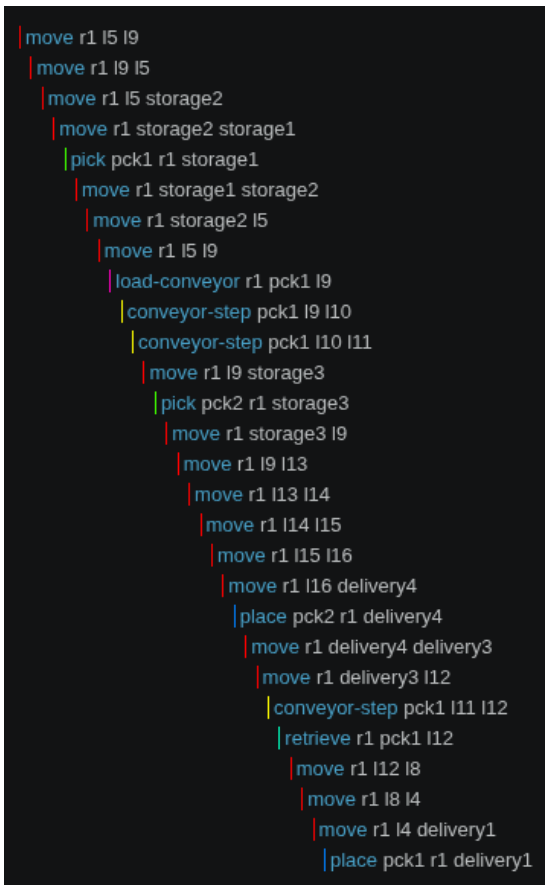

Figure 4: Optional optimal plan

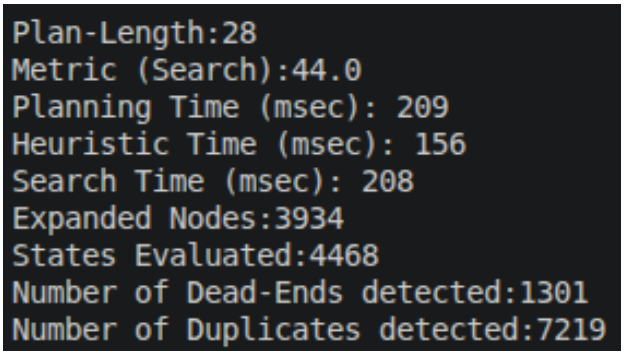

Figure 5: Optional optimal costs

This second solution exploits the conveyor and reduces the metric cost from 54.0 to 44.0, although it requires a larger search effort. This shows that a heuristic planner (ENHSP Planner) may return the first feasible plan before discovering a cheaper conveyor-assisted strategy

3.3 Necessary Conveyor Environment

Compared to the previous scenario, as the Figure 6 shows, several graph connections are removed, so the left storage side and the right delivery side are no longer connected by robot navigation alone. Conveyor use becomes mandatory. Robot r1 operates mainly on the left side, while robot r2 retrieves packages on the delivery side.

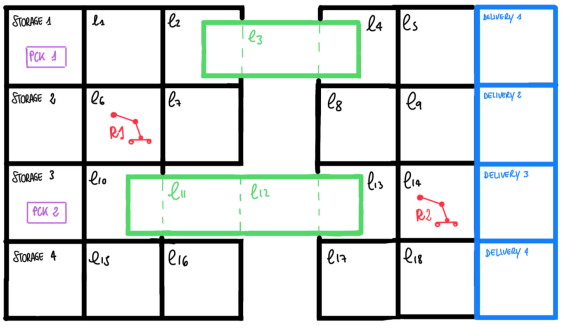

Figure 6: Environment with necessary use of Conveyor Belts

Results Analysis – Necessary Conveyor Environment

The first solution obtained for this scenario has:

Figure 7: Necessary non-optimal plan

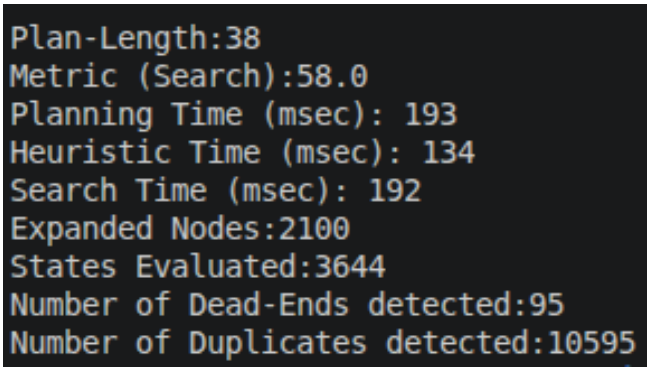

Figure 8: Necessary non-optimal costs

Robot r1 picks packages from the storage areas and loads them onto the conveyor. Then, the packages are advanced along the belt through conveyor-step actions. Robot r2, operating on the right side of the environment, retrieves the packages from the unloading area and places them in the correct delivery zones.

A second, more optimized solution was also obtained:

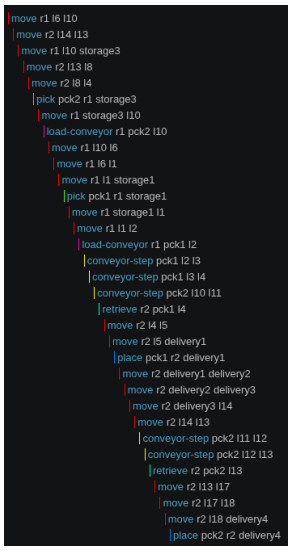

Figure 9: Necessary optimal plan

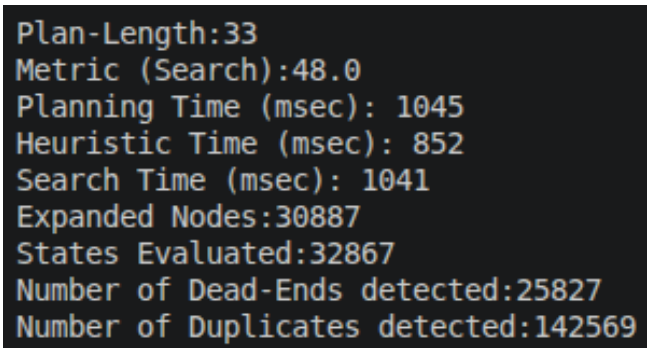

Figure 10: Necessary optimal costs

The cost decreases from 58.0 to 48.0, and the plan length decreases from 38 to 33 actions. The planner achieves this by choosing a better ordering of robot movements, conveyor loading, conveyor advancement, and retrieval actions. However, this improvement requires a much larger computational effort. The number of dead-end increases and the number of expanded nodes increases from 2100 to 30087.

4 Q2 – Advanced PDDL+ Model

In the second part of the project, the warehouse system is extended using PDDL+. Instead of generating conveyor movements as discrete actions, the PDDL+ model introduces continuous processes and instantaneous events. This allows the packages to move automatically along the conveyor belts once they are placed on them, while the robots are only responsible for manipulation, interception, and delivery operations.

4.1 Domain Modelling

The PDDL+ domain is based on four main object types: robot, package, location, and conveyor.

The predicates used in the domain are:

- **belt-segment(?c ?from ?to)**, maps the layout of a specific conveyor belt;
- **conveyor-entry(?c ?l)** / **conveyor-exit(?c ?l)**, which identify the entry and exit points of each belt;
- **conveyor-connected(?c1 ?c2 ?l1 ?l2)**, which allows a package to automatically pass from one conveyor to another;
- **robot-act-on(?r ?l)**, which restricts each robot to the locations it can physically reach, they are not mobile agents;
- **belt-free(?l)**, which prevents two packages from occupying the same conveyor segment;
- **busy(?r)**, which models whether a robot is currently performing an operation.

4.2 Numeric Fluents and Continuous Dynamics

The PDDL+ model introduces numeric fluents to represent time-dependent quantities:

- **(belt-progress ?p)**, which represents the progress of a package along its current conveyor segment;
- **(conveyor-speed ?c)**, which defines the speed of each conveyor;
- **(busy-timer ?r)**, which represents the remaining time before a robot becomes available again;
- **(total-cost)**, which is minimized by the planner.

4.3 Actions, Processes and Events

The domain separates three different types of dynamics. The **Discrete Actions** represent intentional robot operations:

- **load-conveyor**, used by robot1 to place a package onto a conveyor entry point;
- **intercept**, used by intermediate robots (robot 2 to 7) to catch a package while it is moving on a conveyor segment;
- **grasp**, used to retrieve a package from the final segment of a conveyor (used by robot1 at `storage2`);
- **deliver**, used to place a held package into a delivery zone;
- **manipulate-with-package** / **manipulate-no-package**, used when a robot changes its working configuration while holding a package or without carrying a package. These actions carry a heavy execution cost (15) and a long duration (2.0 seconds) to avoid useless motion of the robots;

The **Continuous Processes** represent autonomous background dynamics:

- **package-movement**, which moves packages along conveyor belts. This process continuously increases the progress of a package while it is on a running conveyor:

$$\Delta\text{belt-progress}(p) = \#t \times \text{conveyor-speed}(c)$$

  This process is active as long as the package is on a running conveyor and its progress is less than 1.0. Therefore, once a package is on a belt, it moves automatically over time.

- **robot-timers**, decreases the busy timer of each robot. This models the fact that manipulation operations are not instantaneous: after grasping, delivering, or changing configuration, a robot remains occupied for a certain amount of time before it can execute another action;

The **Instantaneous Events** represent instantaneous state transitions triggered by numeric conditions:

- **advance-segment**, activated when a package reaches a progress ≥ 1.0 on an internal segment. It clears the previous segment, occupies the next one, and resets the progress fluent to 0.0.
- **reach-exit**, activated when a package reaches the end of a conveyor belt handling the automatic physical transition of a package between two connected conveyor belts (e.g., from `cRight` to `cBackR`)
- **free-robot**, automatically resets a robot's `busy` state back to false the moment its `busy-timer` reaches 0.0, freeing its resources for the next action.

4.4 Problem 1: Two-Package Scenario

The first PDDL+ problem considers a relatively simple configuration with two packages. The environment includes seven robots and five conveyor belts operating at a constant speed of 0.8 units/sec.

The initial state defines a sparse distribution:

- **p1** starts on `cRight` at location `l4`;
- **p2** starts on `cMain` at location `l2`.

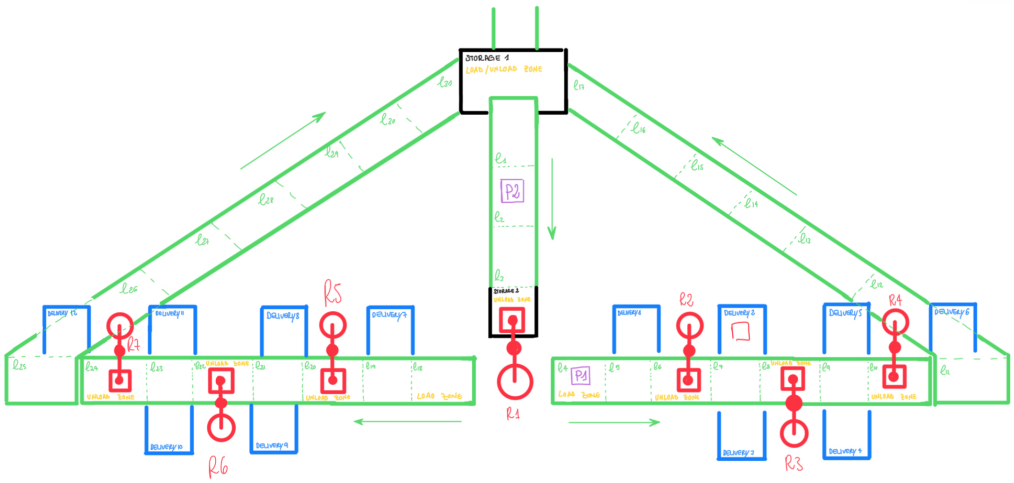

Figure 11: Two-package Scenario

The goal requires delivering `p1` to `delivery2` and `p2` to `delivery12`. This scenario tests whether the planner can coordinate the continuous movement of the conveyors with the manipulation actions of the robots.

```
Found Plan:
0: -----waiting---- [4.0]
4.0: (intercept r2 p1 l6 cRight)
4.0: -----waiting---- [5.0]
5.0: (manipulate-with-package r2 p1 l6 delivery2)
5.0: -----waiting---- [6.0]
6.0: (grasp r1 p2 storage2 cMain)
6.0: -----waiting---- [7.0]
7.0: (manipulate-with-package r1 p2 storage2 l18)
7.0: (deliver r2 p1 delivery2)
7.0: -----waiting---- [9.0]
9.0: (load-conveyor r1 p2 l18 cLeft)
9.0: -----waiting---- [21.0]
21.0: (grasp r7 p2 l24 cLeft)
21.0: -----waiting---- [22.0]
22.0: (manipulate-with-package r7 p2 l24 delivery12)
22.0: -----waiting---- [24.0]
24.0: (deliver r7 p2 delivery12)
```

Figure 12: Two-package scenario plan

```
Plan-Length:43
Elapsed Time: 24.0
Metric (Search):75.0
Planning Time (msec): 52
Heuristic Time (msec): 34
Search Time (msec): 46
Expanded Nodes:34
States Evaluated:52
Number of Dead-Ends detected:4
Number of Duplicates detected:1
```

Figure 13: Two-package scenario costs

The planner coordinates robot operations with autonomous belt motion. Package p1 is intercepted from the right conveyor by robot r2, manipulated to the correct delivery configuration, and delivered to delivery2. Package p2 instead follows a longer path: it reaches storage2, is grasped by r1, transferred to the left conveyor, and later intercepted by r7, which delivers it to delivery12.

The low number of expanded nodes and evaluated states indicates that the search space is still limited in this configuration. Since only two packages are active, the planner has fewer possible conflicts to resolve.

4.5 Problem 2: Four-Package Scenario

The second PDDL+ problem increases the complexity of the task by introducing four packages:

- `p1` and `p2` are closely queued on `cRight` (`l4` and `l5`);
- `p3` and `p4` are placed sequentially on `cMain` (`l1` and `l2`).

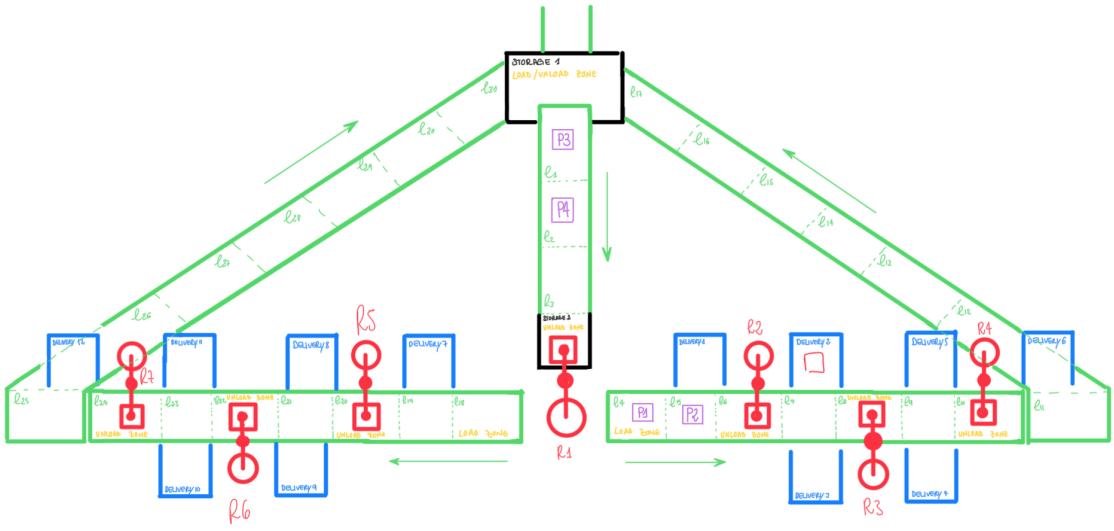

Figure 14: Four-package Scenario

The objective is to route and deliver `p1` and `p2` at `delivery2`, `p3` at `delivery10`, and `p4` at `delivery8`.

Compared with the two-package scenario, the plan is much longer and the computational effort increases dramatically. The plan length grows from 43 to 171 actions, while the number of expanded nodes increases from only 34 to more than 1.3 million.

The main reason for this increase is the interaction between continuous conveyor movement and discrete robot availability. If a robot is busy or in the wrong configuration or takes too long to intercept or grasp an item, the package may continue moving.

```
0: -----waiting---- [5.0]
5.0: (intercept r2 p1 l6 cRight)
5.0: -----waiting---- [7.0]
7.0: (grasp r1 p4 storage2 cMain)
7.0: -----waiting---- [8.0]
8.0: (manipulate-with-package r1 p4 storage2 l18)
8.0: -----waiting---- [10.0]
10.0: (load-conveyor r1 p4 l18 cLeft)
10.0: (manipulate-no-package r1 l18 storage2)
10.0: -----waiting---- [13.0]
13.0: (grasp r1 p3 storage2 cMain)
13.0: -----waiting---- [14.0]
14.0: (manipulate-with-package r1 p3 storage2 l18)
14.0: (intercept r5 p4 l20 cLeft)
14.0: -----waiting---- [15.0]
15.0: (manipulate-with-package r5 p4 l20 delivery8)
15.0: -----waiting---- [17.0]
17.0: (deliver r5 p4 delivery8)
17.0: -----waiting---- [34.0]
34.0: (load-conveyor r1 p3 l18 cLeft)
34.0: (manipulate-no-package r1 l18 storage2)
34.0: -----waiting---- [36.0]
36.0: (grasp r1 p2 storage2 cMain)
36.0: -----waiting---- [37.0]
37.0: (manipulate-with-package r1 p2 storage2 l4)
37.0: (manipulate-with-package r2 p1 l6 delivery2)
37.0: -----waiting---- [39.0]
39.0: (load-conveyor r1 p2 l4 cRight)
39.0: (manipulate-no-package r5 delivery8 l20)
39.0: -----waiting---- [43.0]
43.0: (intercept r6 p3 l22 cLeft)
43.0: -----waiting---- [44.0]
44.0: (manipulate-with-package r6 p3 l22 delivery10)
44.0: (deliver r2 p1 delivery2)
44.0: (manipulate-no-package r2 delivery2 l6)
44.0: -----waiting---- [46.0]
46.0: (deliver r6 p3 delivery10)
46.0: -----waiting---- [75.0]
75.0: (manipulate-no-package r1 l4 storage2)
75.0: -----waiting---- [77.0]
77.0: (grasp r1 p2 storage2 cMain)
77.0: -----waiting---- [78.0]
78.0: (manipulate-with-package r1 p2 storage2 l4)
78.0: -----waiting---- [80.0]
80.0: (load-conveyor r1 p2 l4 cRight)
80.0: -----waiting---- [84.0]
84.0: (intercept r2 p2 l6 cRight)
84.0: -----waiting---- [85.0]
85.0: (manipulate-with-package r2 p2 l6 delivery2)
85.0: -----waiting---- [87.0]
87.0: (deliver r2 p2 delivery2)
```

Figure 15: Four-package scenario plan

```
Plan-Length:171
Elapsed Time: 87.0
Metric (Search):298.0
Planning Time (msec): 177759
Heuristic Time (msec): 154477
Search Time (msec): 177754
Expanded Nodes:1315680
States Evaluated:1724374
Number of Dead-Ends detected:561267
Number of Duplicates detected:1891206
```

Figure 16: Four-package scenario costs

In the two-package case, the planner finds a solution quickly. The system has enough free space on the conveyors, and each package can be handled independently by the assigned robots.

In the four-package case, the same domain becomes much more computationally demanding. The reason is not only the larger number of packages, but also the stronger coupling between them. Packages are not independent anymore: the motion of one package can block or delay another package.

5 Discussion

The two modelling approaches developed in this work highlight the difference between agent-controlled dynamics and environment-driven dynamics. In the basic PDDL model, every relevant change in the world must be explicitly represented as a discrete action selected by the planner.

This is the main approximation introduced in Q1. Each time a package has to move from one conveyor segment to the next one, the planner must explicitly insert a specific action in the plan. Therefore, the conveyor is not truly autonomous: it only moves when the planner decides to apply an action to a package.

This creates important limitations. First, packages on the same conveyor do not move together automatically; each one must be advanced separately. For example, if one package is moved forward with conveyor-step, the other package does not move automatically. The planner has to generate another conveyor-step action for the second package and this makes the model less realistic. Second, the model loses temporal information such as conveyor speed, travel time and the exact instant at which a robot must retrieve an item. As a consequence, the model can only describe the order of actions, but not the continuous evolution of the system over time. Third, package positions are represented only as discrete segment labels, so intermediate motion and interception timing are simplified. In reality, a robot must anticipate the position of the package and act at the correct time. In the basic PDDL model, this anticipation is simplified into reaching the correct discrete location before applying the retrieval action.

The PDDL+ model addresses many of these limitations. In Q2, conveyor motion is represented as an autonomous continuous process. Once a package is placed on a conveyor belt, its progress increases over time according to the conveyor speed. The planner no longer decides every single movement of the package. Instead, it must plan robot actions while anticipating the autonomous evolution of the environment.

This makes the model more realistic. The planner must reason about where a package will be in the future, when it will reach a transfer point, and which robot must be ready to intercept or retrieve it. Therefore, PDDL+ better captures the synchronization problem between robots and autonomous infrastructure.

However, this increased realism also makes the planning problem more complex. The planner must handle both discrete actions and continuous numeric change. In dense scenarios with multiple packages, it must also avoid conflicts between packages sharing the same conveyor segments. This explains why the four-package PDDL+ problem produces a much larger search space than the simpler two-package scenario.

Overall, the comparison between PDDL and PDDL+ shows that classical PDDL is useful for validating the logical structure of the task, such as loading, unloading, and delivery sequences. However, it remains limited when modelling autonomous continuous transport. PDDL+ is more suitable for representing real conveyor belt dynamics, because it separates robot-controlled actions from environment-driven processes and forces the planner to anticipate the autonomous motion of packages.

The results obtained with ENHSP also show an important aspect of heuristic planning. In most of the cases, ENHSP does not immediately return the optimal plan, but rather the first feasible plan found by the search. This is especially visible in the optional and necessary conveyor environment with PDDL. The planner can solve the problem by using only robot navigation, because the conveyor is not required for reachability. This direct solution is easier to find because it follows a simple strategy: the robot picks a package, moves through connected locations, and delivers it. Nevertheless, it is harder for the planner to discover the optimal plan, because it requires additional actions and a less direct sequence: the robot must load the package onto the conveyor, apply conveyor-step actions, retrieve the package at the unloading point, and then complete the delivery. For this reason, the optimal conveyor-assisted plan requires a larger search effort, with more expanded nodes, more evaluated states, and more dead-ends. A heuristic planner may prefer a direct and easily reachable solution, even if another plan with a lower cost exists. To obtain the optimal plan, the planner must explore a larger part of the search space and compare different strategies.